Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification

Functional Pearl

AUTHOR NAME, Institution, Country

Pre- and post-conditions are the standard vocabulary for modular program specification, yet they are silent about obligations that span multiple calls. In real Haskell code such obligations are ubiquitous: a successful TLS handshake must eventually be followed by `bye` before the context is discarded; a taken `MVar` must eventually be put back on *every* code path; a `beginTx` must eventually reach `commit` or `rollback`; a submitted job must complete with probability at least p. No single pre/post pair can express these properties, because the balancing operation lies in an unknown future context.

We present *Pledge*, a Haskell library that augments the classical contract model with *future conditions*: annotations that propagate obligations forward until they are discharged. The `Pledge` monad carries a data-dependent field `future` :: $a \to \mathit{eff}$, allowing obligations to name the specific resource handle an operation returns. Monadic bind partially discharges the first computation's future condition against the continuation's postcondition via *subtraction*; the residual is conjoined with the continuation's own future condition.

The design is parameterised over a `Composable` algebra with concatenation ($\diamond$), conjunction ($\wedge$), subtraction ($\backslash$), and identities `empty` and `universe`. We present four instances: trace protocols via extended regular expressions (RE, with Brzozowski-derivative subtraction and a direct LTL_f embedding); Presburger-guarded traces (GRE, discharged by Z3); semiring-weighted obligations (WRE, covering probabilistic and min-cost scenarios); and separation-logic heap predicates (SL, with magic wand as subtraction). All four share a single bind formula, demonstrating that trace-based, arithmetic, quantitative, and heap-ownership obligations are uniformly expressible within one algebraic interface.

CCS Concepts: • **Software and its engineering → Functional languages**; *Data types and structures*; *Software verification and validation.*

Additional Key Words and Phrases: temporal specification, monadic effects, regular expressions, complement, Brzozowski derivatives, Presburger arithmetic, semiring weights, separation logic, resource lifecycle, Haskell

ACM Reference Format:
Author Name. 2026. Effectful Computations with Future Conditions: A Monadic Approach to Temporal Specification: Functional Pearl. *Proc. ACM Program. Lang.* 1, HASKELL (May 2026), 26 pages.

1 Introduction

Pre- and post-conditions are the workhorse of modular program specification. A pre-condition constrains the state in which an operation may be invoked; a post-condition describes what holds immediately after it returns. Yet a great many correctness properties are not local to a single call boundary: they *span* a sequence of operations, with arbitrary computation in between.

Consider the following three scenarios from the Haskell ecosystem: i) *TLS lifecycle.* A successful `handshake` must eventually be followed by `bye` before the context is discarded. A post-condition on `handshake` records success, but cannot enforce that `bye` will be called, because the discharge

Author's Contact Information: Author Name, Institution, City, Country, author@institution.edu.

ACM 2475-1421/2026/5-ART
https://doi.org/

point lies in an unknown future context. The `bracket` combinator sidesteps this by lexically scoping the context, but cannot express richer message-exchange obligations or handle contexts passed across function boundaries. ii) *MVar discipline.* The `takeMVar` operation removes a value, leaving the variable empty; the caller must restore it with `putMVar` on every exit path. A post-condition records the empty state but cannot require that a future `putMVar` on the *same* variable will occur. iii) *Database savepoints.* The `transactionSave` function opens a fresh transaction that must itself be committed or rolled back; pre/post-conditions on the call cannot carry this obligation forward.

Existing remedies and their limits. The `bracket` function lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. Both `ResourceT` [Snoyman 2011] and `Acquire`/`Managed` thread cleanup actions through every bind but model only the cleanup half of an obligation, discharging it uniformly at scope exit rather than by application-specific operations. Linear Haskell [Bernardy et al. 2018] enforces "consume exactly once" but cannot express branching discharge obligations or quantitative commitments. Parameterised and indexed monads [Atkey 2009] encode protocol states at the type level but require `RebindableSyntax` or `QualifiedDo`, produce opaque type errors, and are confined to a single domain per design.

Our approach. We propose the `Pledge` monad as a lightweight, compositional mechanism for specifying such obligations, requiring no language extensions beyond standard Haskell type classes. A value of type `Pledge eff a` carries three annotations: a precondition `pre` $::$ *eff*, a postcondition `post` $::$ *eff*, and a data-dependent future condition `future` $:: a \to$ *eff*. Because `future` ranges over the return value, an obligation can name the actual resource handle produced at runtime: `tlsHandshake` can declare that *whichever* `Context` it returns must eventually receive `bye`. Monadic bind partially discharges the first computation's future via `subtraction` against the continuation's postcondition; the residual is conjoined with the continuation's own future condition. The user annotates individual operations only; the monad propagates obligations automatically.

The `Composable` typeclass abstracts the specification algebra into five operations: concatenation ($\diamond$), conjunction ($\wedge$), subtraction ($\backslash$), and their respective identities `empty` and `universe`. It admits four independent instances:

- *(Trace protocol,* RE*)* Extended regular expressions over events, with complement as a first-class constructor. Subtraction is implemented via Brzozowski derivatives; LTL_f operators embed directly as RE terms without automaton construction. This instance handles `takeMVar`/`putMVar` discipline, TLS handshake/bye pairing, transaction lifecycle, and lock protocols.
- *(Arithmetic bounds,* GRE*)* Each RE is paired with a Presburger arithmetic predicate over program variables, checked by an SMT solver (Z3 via SBV). A single annotation simultaneously enforces a protocol ordering *and* a numeric bound, catching, for example, counter overflow before any execution takes place.
- *(Quantitative obligation,* WRE*)* Boolean membership is generalised to a semiring weight. The probability semiring captures obligations of the form "this job completes with probability at least p"; the tropical semiring gives minimum-cost scheduling obligations. A weight of `szero` in the residual signals an unfulfilled quantitative commitment.
- *(Heap ownership,* SL*)* Separation-logic heap predicates, with separating conjunction as sequential composition and magic wand as subtraction. Future conditions describe what heap state must hold when the computation completes; a leaked `Cell` in the residual post names the unreleased ownership precisely.

Contributions.

(1) We define the `Pledge` monad (§3), parameterised over a `Composable` algebra, with `pre` :: *eff*, `post` :: *eff*, and a data-dependent `future` :: $a \to$ *eff* field, and verify that the monad laws hold for the (*ret*, *post*, *future*) triple (§9).
(2) We instantiate the algebra to extended regular expressions over events with complement, using Brzozowski derivatives to implement subtraction (§2.1), and show that LTL_f operators embed directly as RE terms (§2.3).
(3) We extend the RE instance to a guarded form (GRE, §5), pairing each RE with a Presburger arithmetic predicate over program variables, evaluated by an SMT solver (Z3 via SBV) at membership-check time.
(4) We provide a weighted RE instance (WRE, §6), parameterised over a semiring, that generalises Boolean membership to a quantitative weight, recovering probabilistic and min-cost future conditions as special cases.
(5) We provide a fourth, independent instantiation to separation-logic heap predicates (§7), demonstrating that the `Composable` interface extends beyond trace-based specifications.
(6) We demonstrate the approach on ten use cases across all four instances (§8), showing that resource leaks, ordering violations, arithmetic overflows, probabilistic obligations, and heap-ownership errors all surface as non-trivial residual annotations.

2 Background

2.1 Extended Regular Expressions over Events

Events are the atomic units of observation; each carries a name and a list of typed arguments.

```
data Term  = Str String | Num Int | List [Term]
data Event = Atom String [Term]   -- concrete event, e.g. Atom "send" [Num 1]
           | Wildcard             -- pattern matching any single event
data RE = Bot | Epsilon | Single Event
        | Seq RE RE | Or RE RE | And RE RE | Star RE | Not RE
```

The specification language is an extended regular expression type that adds complement as a first-class constructor:

$$r \;::=\; \varnothing \mid \varepsilon \mid e \mid r_1 \cdot r_2 \mid r_1 \vee r_2 \mid r_1 \wedge r_2 \mid r^* \mid \neg r$$

The constructors are standard. The empty language $\varnothing$ (**`Bot`**) accepts no trace. The empty-trace language ε (**`Epsilon`**) accepts only the empty sequence. A single-event term e (**`Single`** `Event`) matches exactly one event; **`Wildcard`** is a pattern that matches any single event. Concatenation $r_1 \cdot r_2$ (**`Seq`**) accepts a trace if it splits into a prefix in $L(r_1)$ followed by a suffix in $L(r_2)$. Union $r_1 \vee r_2$ (**`Or`**) accepts traces in either language. The Kleene star r^* (**`Star`**) accepts zero or more repetitions of traces from $L(r)$; Σ^* is the set of all finite event sequences over the alphabet Σ (the universal language). Intersection $r_1 \wedge r_2$ (**`And`**) is derived as $\neg(\neg r_1 \vee \neg r_2)$ via De Morgan. Complement $\neg r$ (**`Not`**) is the set of all traces in Σ^* *not* in $L(r)$. Moreover, we use the following five derived forms throughout:

$$\begin{aligned}
\texttt{anything} &\triangleq \neg\varnothing && \text{(the universal language } \Sigma^*\text{)}\\
\texttt{finally}(e) &\triangleq \texttt{anything} \cdot e \cdot \texttt{anything} && (e \text{ must occur eventually})\\
\texttt{globally}(e) &\triangleq e^* && (e \text{ at every step})\\
\texttt{never}(e) &\triangleq \neg\texttt{finally}(e) && (e \text{ must never occur})\\
\texttt{noUntil}(e, g) &\triangleq \neg\big((\Sigma \setminus \{g\})^* \cdot e \cdot \texttt{anything}\big) && (e \text{ must not occur before } g)
\end{aligned}$$

`noUntil`$(e,\ g)$ is nullable (the empty continuation satisfies it) and its derivative with respect to g is Σ^*, meaning once g occurs the restriction on e is lifted. This is the key combinator for *conditional* safety: e is forbidden until g re-enables it.

2.2 Algebraic Complement and Brzozowski Derivatives

The Brzozowski derivative [Brzozowski 1964] $\partial_a(r)$ computes the language of continuations after reading event a. The key law for complement is:

$$\partial_a(\neg r) \;=\; \neg\big(\partial_a(r)\big) \tag{1}$$

The nullability function $\nu(r)$ (= `True` iff $\varepsilon \in L(r)$) satisfies $\nu(\neg r) = \neg\,\nu(r)$. Together, equations (1) and the De Morgan laws

$$\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2 \qquad \neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$$

allow complement to be propagated and simplified *purely algebraically*, without building a DFA. The normaliser additionally applies $\neg\neg r = r$, $\neg\varnothing = \neg\varnothing$, and $\neg\neg\varnothing = \varnothing$. We define the nullable and derivative functions in Haskell as follows:

```
nullable :: RE → Bool
nullable (Not r)     = not (nullable r)      -- nu(Not r) = not(nu(r))
nullable (Star _)    = True
nullable (Seq r1 r2) = nullable r1 && nullable r2
nullable (Or  r1 r2) = nullable r1 || nullable r2
nullable (And r1 r2) = nullable r1 && nullable r2
nullable Epsilon     = True
nullable _           = False

derivative :: Event → RE → RE
derivative e (Not r)      = Not (derivative e r)   -- d_a(Not r) = Not(d_a(r))
derivative e (Single p)   = if matchEvent e p then Epsilon else Bot
derivative e (Seq r1 r2)
    | nullable r1 = Or (Seq (derivative e r1) r2) (derivative e r2)
    | otherwise   = Seq (derivative e r1) r2
derivative e (Or  r1 r2)  = Or  (derivative e r1) (derivative e r2)
derivative e (And r1 r2)  = And (derivative e r1) (derivative e r2)
derivative e (Star r)     = Seq (derivative e r) (Star r)
derivative _ _            = Bot
```

The `firstWith` function returns the set of events (drawn from a supplied finite alphabet Σ_0) that can begin a word in $L(r)$, driving the subtraction algorithm. The complement case requires particular care. Naively writing `first (Not r) = first r` is *unsound*: $L(\neg r)$ can begin with events *not* in $\mathit{first}(r)$. The correct characterisation is:

$$e \in \mathit{first}(\neg r, \Sigma_0) \iff e \in \Sigma_0 \;\wedge\; \lceil \partial_e(r) \rceil \neq \neg\varnothing \tag{2}$$

That is, e can open a word in $L(\neg r)$ if and only if, after consuming e, the residual $\partial_e(r)$ does *not* cover the entire language: there exists some continuation that remains outside $L(r)$. The condition $\lceil \partial_e(r) \rceil \neq \neg\varnothing$ checks exactly this. If the normalised derivative equals the universal language Σ^*, then every continuation after e belongs to $L(r)$; consequently, no continuation after e belongs to $L(\neg r)$, and e cannot open any word in that language.

```
atoms :: RE → [Event]
atoms (Single Wildcard) = []
atoms (Single e)        = [e]
atoms (Seq r1 r2)       = atoms r1 `union` atoms r2
atoms (Or  r1 r2)       = atoms r1 `union` atoms r2
atoms (And r1 r2)       = atoms r1 `union` atoms r2
atoms (Star r)          = atoms r
atoms (Not r)           = atoms r
atoms _                 = []

firstWith :: [Event] → RE → [Event]
firstWith _    Bot              = []
firstWith _    Epsilon          = []
firstWith _    (Single e)       = [e]
firstWith alph (Seq r1 r2)
    | nullable r1               = firstWith alph r1 `union` firstWith alph r2
    | otherwise                 = firstWith alph r1
firstWith alph (Or  r1 r2)      = firstWith alph r1 `union` firstWith alph r2
firstWith alph (And r1 r2)      = [e | e <- firstWith alph r1,
                                       e `elem` firstWith alph r2]
firstWith alph (Star r)         = firstWith alph r
firstWith alph (Not r)          = [e | e <- alph,
                                       not (isTotal (normalize (derivative e r)))]
  where isTotal (Not Bot) = True; isTotal _ = False
```

The alphabet Σ_0 is collected from an RE via `atoms`, which recursively gathers all concrete (**`Atom`**) events, excluding **`Wildcard`**. The function `first r = firstWith (atoms r) r` is a convenient wrapper for standalone use; inside `subtraction` both operands contribute to the alphabet (see §4).

Illustrative cases. Two small examples fix the intuition for Equation (2). When $r = a \cdot \Sigma^*$ and $\Sigma_0 = \{a, b\}$, the naive rule would return $\{a\}$, but $L(\neg r)$ consists of words beginning with events other than a: $\partial_a(a \cdot \Sigma^*)$ normalises to $\neg\varnothing$, excluding a, while $\partial_b(a \cdot \Sigma^*) = \varnothing \neq \neg\varnothing$ includes b, giving the correct $\{b\}$. When $r = \varepsilon$ and $\Sigma_0 = \{a, b\}$, the naive rule returns $\varnothing$ (since $\mathit{first}(\varepsilon) = \varnothing$), but $\neg\varepsilon$ accepts all non-empty traces: $\partial_a(\varepsilon) = \varnothing \neq \neg\varnothing$ includes a, and likewise b, giving $\{a, b\}$.

Table 1. Selected `firstWith` evaluations with **`Not`**. Σ_0 is the explicitly supplied alphabet. Rows marked † produce a *strict* subset of Σ_0.

RE r	Σ_0	`firstWith` Σ_0 $(\neg r)$	Rationale
a	$\{a, b\}$	$\{a, b\}$	$\partial_a(a) = \varepsilon$, $\partial_b(a) = \varnothing$; neither Σ^*
$a \cdot b$	$\{a, b, c\}$	$\{a, b, c\}$	all derivatives non-Σ^*
$a \vee b$	$\{a, b, c\}$	$\{a, b, c\}$	$\partial_c(a \vee b) = \varnothing$, still non-$\Sigma^*$
a^*	$\{a, b\}$	$\{a, b\}$	$\partial_a(a^*) = a^*$, non-Σ^*; b starts $[b]$
$\neg a$ (double neg.)	$\{a, b\}$	$[a]$	$\partial_b(\neg a) = \Sigma^* \Rightarrow b$ excluded†
$a \cdot \Sigma^*$	$\{a, b\}$	$[b]$	$\partial_a(a \cdot \Sigma^*) = \Sigma^* \Rightarrow a$ excluded†
$a \cdot \Sigma^* \vee b \cdot \Sigma^*$	$\{a, b, c\}$	$[c]$	both a, b excluded; only c avoids†
$\neg a \wedge \neg b$	$\{a, b, c\}$	$\{a, b\}$	De Morgan: $= \neg(a \vee b)$; ∂_c is Σ^*†

Table 1 extends these to seven further patterns. The first four rows confirm that complement does not exclude any alphabet event when the derivative is non-total. The double-negation row shows correct restoration of the original first set: $\partial_b(\neg a) = \Sigma^*$ is total, so b is filtered out. In $\neg(a \cdot \Sigma^*)$, $\partial_a(a \cdot \Sigma^*)$ normalises to Σ^*, excluding a; only b survives. Similarly, $\neg(a \cdot \Sigma^* \vee b \cdot \Sigma^*)$ excludes both a and b, leaving only c, while the De Morgan row verifies that $\neg(\neg a \wedge \neg b) = a \vee b$ yields $\{a, b\}$.

2.3 Embedding $\mathrm{LTL_f}$

Because complement is a first-class constructor, the standard $\mathrm{LTL_f}$ operators translate directly into RE, with no automaton construction needed. Table 2 gives the full mapping.

Table 2. $\mathrm{LTL_f}$ to RE translation. $\Sigma^1 \triangleq$ `Single Wildcard`, $\neg\varnothing \triangleq \neg\varnothing$.

LTL formula	RE $[\![\cdot]\!]$	
$\top$	$\neg\varnothing$	
$\bot$	$\varnothing$	
e	$\{e\}$	
$\neg\phi$	$\neg[\![\phi]\!]$	
$\phi \wedge \psi$	$[\![\phi]\!] \cap [\![\psi]\!]$	
$\phi \vee \psi$	$[\![\phi]\!] \cup [\![\psi]\!]$	
$X\,\phi$	$\Sigma^1 \cdot [\![\phi]\!]$	
$F\,\phi$	$\neg\varnothing \cdot [\![\phi]\!]$	
$G\,\phi$	$\neg(\neg\varnothing \cdot \neg[\![\phi]\!])$	
$\phi\,U\,\psi$	$\mathit{step}(\phi)^* \cdot [\![\psi]\!]$	(ϕ propositional)

The complement operator does the heavy lifting: $G\,\phi \triangleq \neg F \neg\phi$ unfolds to $\neg(\neg\varnothing \cdot \neg[\![\phi]\!])$ using two applications of $\neg$; $F\,\phi$ and $X\,\phi$ need only concatenation. The U case uses $\mathit{step}(\phi)$, the length-1 RE for a single event satisfying ϕ, which is defined only when ϕ is propositional.

3 The `Pledge` Monad

3.1 The `Composable` Class

We abstract the specification algebra via a type class:

```
class Composable a where
  concatenation :: a → a → a   -- (⋄), unit: empty
  conjunction   :: a → a → a   -- (∧), unit: universe
  empty         :: a           -- identity for ⋄  (varepsilon)
  universe      :: a           -- identity for ∧  (neg-emptyset)
  subtraction   :: a → a → a   -- (\)
```

The five operations form exactly the interface required by the bind formula. Here `concatenation` sequences effects; `conjunction` combines simultaneous constraints. The identities `empty` and `universe` appear in `pure`. In particular, the operation `subtraction r1 r2` computes the residual of the *second* argument with respect to the first: it asks "which part of the obligation r_2 is not yet discharged, given that a trace matching r_1 has been observed?" The first argument r_1 is the *observed* trace (a postcondition); the second argument r_2 is the *pending obligation* (a precondition or future condition to be discharged). Two base cases fix the endpoints: $\varepsilon \setminus r_2 = r_2$ (nothing observed, the full obligation r_2 remains) and $\Sigma^* \setminus r_2 = \varepsilon$ (everything observed, the obligation is fully discharged). The general

case advances both r_1 and r_2 in lockstep: for each event e that r_1 can begin with, the residual after e is $\partial_e(r_1) \setminus \partial_e(r_2)$, consuming one event from each side simultaneously.

Quick example. Let the postcondition $r_1 = \texttt{putMVar}(v)$ (`putMVar(v)` just occurred) and the obligation $r_2 = \texttt{finally}(\texttt{putMVar}(v))$ (`putMVar(v)` must eventually occur). The event $\texttt{putMVar}(v)$ in r_1 satisfies $\texttt{finally}(\texttt{putMVar}(v))$, so the residual $r_1 \setminus r_2 = \neg\varnothing$ and the obligation is fully discharged. Conversely, if $r_1 = \texttt{send}(v)$ instead, then $\texttt{send}(v) \setminus \texttt{finally}(\texttt{putMVar}(v)) = \texttt{finally}(\texttt{putMVar}(v))$: a `send` event makes no progress toward the `putMVar` obligation, leaving it intact.

3.2 The `Pledge` Type

The `future` field is *data-dependent*: it is a function of the return value `a`, allowing obligations to refer to the specific resource handle that an operation produces. The helper `evalFuture` evaluates this function at the computation's own return value.

```
data Pledge eff a = Pledge
  { ret    :: a
  , pre    :: eff        -- what must have occurred before
  , post   :: eff        -- what this computation produces
  , future :: a → eff    -- obligation indexed by the return value
  }

evalFuture :: Pledge eff a → eff
evalFuture e = future e (ret e)
```

3.3 Monad Instances

The `pure` function introduces a value with no effect and no obligation:

```
pure x = Pledge { ret = x, pre = universe, post = empty, future = \_ → universe }
```

The bind operator sequences two computations; write $e_2 = f(ret(e))$.

```
e >>= f = let fe = f (ret e) in Pledge
  { ret    = ret fe
  , pre    = pre e ∧ (post e \\ pre fe)
  , post   = post e ⋄ post fe
  , future = \_ → (post fe \\ future e (ret e)) ∧ future fe (ret fe)
  }
```

Future condition. Each `future` function is applied to the corresponding `ret` before the obligation is propagated:

$$future(e \gg\!= f) \;=\; \big(post(e_2) \setminus future(e)(ret(e))\big) \;\wedge\; future(e_2)(ret(e_2)) \tag{3}$$

This removes from e's future obligation (evaluated at e's return value) those traces that e_2's postcondition satisfies, then intersects the residual with e_2's own future obligation (evaluated at e_2's return value). When the result normalises to $\neg\varnothing$, all obligations are discharged.

Precondition.

$$pre(e \gg\!= f) \;=\; pre(e) \;\wedge\; \big(post(e) \setminus pre(e_2)\big) \tag{4}$$

The term $post(e) \setminus pre(e_2)$ is the *residual precondition*: the part of e_2's precondition not already discharged by e's postcondition. Both must hold simultaneously in the same input history.

When $post(e)$ does not cover $pre(e_2)$, the residual is $\varnothing$, flagging a violation. When it fully covers $pre(e_2)$, the residual is ε and the combined precondition reduces to $pre(e)$ (the standard Hoare sequencing rule).

3.4 The Applicative Instance

The `Pledge` monad also satisfies the `Applicative` class, which provides `<*>` for combining a function-valued computation with an argument-valued computation *without* data-dependency between them. Writing e_f for the function computation and e_x for the argument computation:

```
ef <*> ex = Pledge
  { ret    = ret ef (ret ex)
  , pre    = pre ef ∧ (post ef \\ pre ex)
  , post   = post ef ⋄ post ex
  , future = \_ → (post ex \\ future ef (ret ef)) ∧ future ex (ret ex)
  }
```

The structure mirrors $\gg=$, i.e., preconditions and postconditions compose in the same way, and the future condition is the conjunction of the two evaluated futures after subtracting the continuation's postcondition from the first computation's future. The key difference is that `<*>` cannot make e_x *depend* on the return value of e_f; $\gg=$ passes $ret(e)$ into f, enabling the data-dependent future conditions exemplified by `tlsHandshake` and `takeMVar`.

3.5 Intuition via an Example

The `takeMVar` operation returns the handle of the taken `MVar` as its return value; the future condition is data-dependent, referring to whatever handle was actually returned. For `putMVar`, the precondition requires that `takeMVar(v)` has occurred previously in the trace, and the future asserts that `putMVar(v)` must not recur until `takeMVar(v)` is called again, preventing double-put.

```
type MVar = Int

takeMVar :: MVar → Pledge RE MVar
takeMVar v = Pledge
  { ret = v
  , pre = universe
  , post   = Single (Atom "takeMVar" [Num v])
  , future = \r → finally (Atom "putMVar" [Num r]) }

putMVar :: MVar → Pledge RE ()
putMVar v = Pledge
  { ret = ()
  , pre    = previously (Atom "takeMVar" [Num v])
  , post   = Single (Atom "putMVar" [Num v])
  , future = \_ → noUntil (Atom "putMVar" [Num v]) (Atom "takeMVar" [Num v]) }
```

3.5.1 *A Good Program:* `v <- takeMVar 1; putMVar v`.

After `takeMVar 1` ($ret = 1$) the annotations are:

$$post = \texttt{takeMVar}(1), \qquad future = \texttt{finally}(\texttt{putMVar}(1)), \qquad pre = \neg\varnothing.$$

Binding `putMVar 1` (with $pre_2 = \texttt{previously}(\texttt{takeMVar}(1))$, $post_2 = \texttt{putMVar}(1)$):

Precondition.

$$\texttt{takeMVar}(1) \setminus \texttt{previously}(\texttt{takeMVar}(1)) = \neg\varnothing,$$

so $pre = \neg\varnothing \wedge \neg\varnothing = \neg\varnothing$. ✓

Future.

$$\texttt{putMVar}(1) \setminus \texttt{finally}(\texttt{putMVar}(1)) = \neg\varnothing;$$

conjoined with $future_2 = \texttt{noUntil}(\texttt{putMVar}(1), \texttt{takeMVar}(1))$ gives $\texttt{noUntil}(\texttt{putMVar}(1), \texttt{takeMVar}(1))$. ✓

Note that the residual $\texttt{noUntil}(\texttt{putMVar}(1), \texttt{takeMVar}(1))$ is nullable, so the program may terminate here with all obligations met. The formula is a safety guard only: a second `putMVar(1)` is forbidden until `takeMVar(1)` resets it; no positive obligation requires another `putMVar`.

3.5.2 A Bad Program: `putMVar 1` *alone.*
No preceding `takeMVar` means $post = \varepsilon$, so:

$$\varepsilon \setminus \texttt{previously}(\texttt{takeMVar}(1)) = \texttt{previously}(\texttt{takeMVar}(1)),$$

which is not nullable. The combined precondition $pre = \texttt{previously}(\texttt{takeMVar}(1)) \neq \neg\varnothing$ signals the violation.

3.5.3 A Bad Program: `takeMVar 1 ≫ takeMVar 2 ≫ putMVar 1`.
Only the obligation for `MVar 1` is discharged. The future of `takeMVar 2`, namely $\texttt{finally}(\texttt{putMVar}(2))$, is untouched, so the residual retains $\texttt{finally}(\texttt{putMVar}(2))$, naming the unreleased variable precisely.

3.5.4 A Bad Program: `v <- takeMVar 1; putMVar v; putMVar v`.
After the first `putMVar 1` the future is:

$$\texttt{noUntil}(\texttt{putMVar}(1), \texttt{takeMVar}(1)).$$

The second `putMVar 1` has $post_2 = \texttt{putMVar}(1)$; subtracting from the `noUntil` formula yields $\varnothing$, so the future collapses to $\varnothing$.

4 The RE Instance

The `Composable RE` instance maps the algebra operations onto `RE` constructors and implements subtraction via derivatives:

```
instance Composable RE where
  concatenation = Seq
  conjunction   = And
  empty         = Epsilon
  universe      = anything      -- = Not Bot

  subtraction Epsilon   r2 = r2
  subtraction (Not Bot) _  = Epsilon   -- Sigma* \\ r = eps (trivially satisfied)
  subtraction r1 r2 =
    let alph   = atoms r1 `union` atoms r2   -- combined alphabet
        evts   = firstWith alph r1
        step e = subtraction (normalize (derivative e r1))
                             (normalize (derivative e r2))
    in foldr Or Bot (map step evts)

normalize :: RE → RE   -- RE-specific; not part of Composable
```

```
normalize r = {- ... algebraic simplification ... -}
```

The second clause handles the universal case: $\neg\varnothing \setminus r = \varepsilon$ for any r. The general clause collects the alphabet from *both* operands before calling `firstWith`, ensuring events appearing only in r_2 are still considered under complement via Equation (2). The `normalize` function is defined outside the instance and applies the RE-specific laws listed in Table 3, which exploit the complement constructor.

Table 3. Normalisation laws applied by `normalize`. All rules are applied recursively bottom-up.

Law	Constructor
$\varepsilon \cdot r = r,\ r \cdot \varepsilon = r$	`Seq`
$\varnothing \vee r = r,\ r \vee \neg\varnothing = \neg\varnothing$	`Or`
$\varnothing \wedge r = \varnothing,\ r \wedge \neg\varnothing = r$	`And`
$\neg\neg r = r$	`Not`
$\neg(r_1 \vee r_2) = \neg r_1 \wedge \neg r_2$	`Not`
$\neg(r_1 \wedge r_2) = \neg r_1 \vee \neg r_2$	`Not`
$\neg\varnothing = \neg\varnothing,\ \neg\neg\varnothing = \varnothing$	`Not`
$\varnothing^* = \varepsilon,\ \varepsilon^* = \varepsilon$	`Star`

5 The GuardedRE Instance

The RE instance reasons about event traces but is entirely silent about arithmetic state. The GRE instance closes this gap: a guarded regular expression pairs a trace RE with a Presburger arithmetic predicate over a concrete heap, so that a single annotation can simultaneously enforce a protocol ordering *and* an arithmetic bound.

5.1 The GuardedRE Type

```
data GuardedRE = GuardedRE PPred RE
    deriving (Eq)
```

A state (heap, trace) satisfies `GuardedRE p r` iff

$$\text{heap} \models p \quad \wedge \quad \text{trace} \in L(r).$$

The two sides are independent: p is a static constraint on the heap, while r advances through Brzozowski derivatives as events arrive.

Two smart constructors lift a pure RE or a pure Presburger predicate:

```
fromRE    :: RE    → GuardedRE
fromRE r   = GuardedRE PTrue r        -- no heap constraint

fromPPred :: PPred → GuardedRE
fromPPred p = GuardedRE p universe    -- accept any trace
```

5.2 The `Composable GuardedRE` Instance

```
instance Composable GuardedRE where
    concatenation (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (Seq r1 r2))
```

```
    conjunction (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2)) (normalize (And r1 r2))
    empty    = GuardedRE PTrue Epsilon
    universe = GuardedRE PTrue (Not Bot)   -- = anything
    subtraction (GuardedRE p1 r1) (GuardedRE p2 r2) =
        GuardedRE (normalizePPred (PAnd p1 p2))
                  (normalize (reSubtraction r1 r2))
```

Both `concatenation` and `subtraction` conjoin both heap predicates: the residual must respect the constraints from both operands. The RE component uses the same `reSubtraction` and `normalize` as the plain RE instance.

5.3 Presburger Normalisation

Composing multiple **`GuardedRE`** values accumulates a tree of `PAnd` nodes. The `normalizePPred` function flattens this tree, removes duplicates, substitutes equalities (e.g. $h[a] = k \Rightarrow$ replace `ValAt` a by k throughout), evaluates literal comparisons, and replaces $\neg$**true** with the canonical **false** constructor **`PFalse`**:

```
data PPred
    = PTrue | PFalse      -- canonical true/false
    | PLt PExpr PExpr | ...
    | PNot PPred | PAnd PPred PPred

normalizePPred :: PPred → PPred
normalizePPred p = rebuildAnd (substEqs eqs atoms)
  where
    atoms = flattenAnd (normStep p)
    eqs   = [ (a, k) | PEq (ValAt a) (Lit k) <- atoms ]
```

The introduction of **`PFalse`** prevents the solver from being invoked with expressions such as ($\neg$**true**) $\wedge \cdots$ that are syntactically unsatisfiable; these normalise immediately to **`PFalse`** without any solver call.

5.4 Example: Bounded Counter

A bounded counter stored at heap address a with maximum value M must follow the protocol `init` $\cdot$ (`inc` | `dec`)* $\cdot$ `snapshot`. Increment is additionally guarded by $h[a] < M$ (no overflow); decrement by $h[a] > 0$ (no underflow).

```
initCounter :: Addr → Pledge GuardedRE ()
initCounter a = Pledge
  { ret = ()
  , pre  = fromRE universe
  , post = GuardedRE (PEq (ValAt a) (Lit 0))
                     (Single (Atom "init" [Num a]))
  , future = \_ → fromRE universe }

increment :: Addr → Int → Pledge GuardedRE ()
increment a maxVal = Pledge
  { ret = ()
  , pre  = GuardedRE (PLt (ValAt a) (Lit maxVal)) universe
```

```
  , post = fromRE (Single (Atom "inc" [Num a]))
  , future = \_ → fromPPred (PGe (ValAt a) (Lit 0)) }
```

The `pre` of `increment` carries $h[a] < M$ (pure Presburger) conjoined with the universal RE. After composing eleven increments against a bound of 10, `normalizePPred` derives the contradiction $h[a] < 10 \wedge \neg(h[a] < 10) \equiv \mathbf{false}$, flagging the overflow before any execution takes place.

Bad programs and residuals.

Bad program	Residual `pre` (Presburger)	Residual `pre` (RE)
`overflow` (11 incs, max=10)	**false**	$\neg\varnothing$
`underflow` (dec at zero)	**false**	$\neg\varnothing$
`noInit` (skip init)	$\neg\varnothing$	$\varnothing$

6 The WeightedRE Instance

The RE and GRE instances are Boolean: a future condition is either satisfied or violated. The WRE instance generalises membership to a *weight* drawn from a semiring, enabling probabilistic and quantitative obligations to be expressed within the same `Composable` framework.

6.1 The Semiring Class

```
class (Eq w, Show w) ⇒ Semiring w where
    szero :: w          -- additive identity  (blocked path)
    sone  :: w          -- multiplicative identity
    sadd  :: w → w → w
    smul  :: w → w → w
```

Setting w = `Bool` (with `sadd` = $(\vee)$, `smul` = $(\wedge)$) recovers the Boolean RE instance. Two further instances capture quantitative scenarios:

Instance	`szero`	`sadd`	`smul`
`Prob` (probability)	0	$\min(p_1 + p_2,\ 1)$	$p_1 \times p_2$
`Tropical` (min-cost)	$+\infty$	min	$+$

In the probability semiring, `sadd` is bounded addition (sum of probabilities capped at 1) and `smul` is multiplication. In the tropical semiring, `sadd` is minimum (taking the cheaper alternative) and `smul` is addition (summing costs along a path).

6.2 The `WRE` Type and Operators

```
data WRE w
    = WBot              -- weight szero: no accepting path
    | WEps w            -- weight w: accept the empty trace
    | WSingle w Event   -- weight w: accept exactly this event
    | WSeq (WRE w) (WRE w)
    | WAdd (WRE w) (WRE w)
    | WAnd (WRE w) (WRE w)
    | WStar (WRE w)
```

The generalised nullable function accumulates the weight of all accepting paths through the empty trace:

```
wNullable :: Semiring w ⇒ WRE w → w
wNullable (WEps w)      = w
wNullable (WSeq r1 r2)  = smul (wNullable r1) (wNullable r2)
wNullable (WAdd r1 r2)  = sadd (wNullable r1) (wNullable r2)
wNullable (WAnd r1 r2)  = smul (wNullable r1) (wNullable r2)
wNullable (WStar r)     = sone
wNullable _             = szero
```

The functions `wDerivative` and `wSubtraction` follow the same structure as their Boolean counterparts, with semiring operations substituted for Boolean ones. The `Composable (WRE w)` instance (enabled by `FlexibleInstances`) maps `concatenation` to `WSeq`, `conjunction` to `WAnd`, and `subtraction` to `wSubtraction`.

6.3 Derived Forms

```
wTop     :: Semiring w ⇒ WRE w
wTop      = WStar (WSingle sone Wildcard)   -- sone-weighted anything

wFinally :: Semiring w ⇒ w → Event → WRE w
wFinally weight e = WSeq wTop (WSingle weight e)

wGlobally :: Semiring w ⇒ w → Event → WRE w
wGlobally weight e = WStar (WSingle weight e)
```

6.4 Example: Probabilistic Memory (Prob Semiring)

In a probabilistic setting, each `free(a)` has an associated probability of being called. Each `malloc` call carries a future condition asserting that `free(a)` must be called with probability at least p:

```
malloc :: Addr → Double → Pledge (WRE Prob) Addr
malloc a p = Pledge
  { ret = a, pre = wTop
  , post   = WSingle sone (Atom "malloc" [Num a])
  , future = \r → wFinally (Prob p) (Atom "free" [Num r]) }
```

The residual weight of `wNullable` at the end of a program measures how much of the obligation has been satisfied: a weight of `Prob 1.0` indicates that the obligation is fully discharged; a weight strictly less than 1 quantifies the probability shortfall.

6.5 Example: Min-Cost Scheduling (Tropical Semiring)

Under the tropical semiring, `WSingle (Tropical c) e` assigns cost c to emitting event e; `wNullable` then returns the minimum total cost of reaching an accepting state:

```
submitJob :: Int → Double → Pledge (WRE Tropical) ()
submitJob jobId cost = Pledge
  { ret = ()
  , pre  = wTop
  , post = WSingle (Tropical cost) (Atom "submit" [Num jobId])
  , future = \_ →
      wFinally (Tropical 0) (Atom "complete" [Num jobId]) }
```

Composing several `submitJob` calls yields a future whose `wNullable` is the minimum total cost of completing all submitted jobs. A program that omits a `complete` event for some job i retains `wFinally (Tropical 0) (complete i)` in the residual, and the overall weight is $+\infty$ (i.e. `szero`), flagging the missing completion.

7 The SL Instance

The `Composable` typeclass is not inherently tied to trace-based specifications. We demonstrate its generality with a fourth instance over *separation-logic heap predicates*, where future conditions describe what heap state must hold upon completion of the computation.

7.1 Structural Parallel with RE

Table 4 shows the role each SL construct plays in the `Composable` interface.

Table 4. Structural parallel between the four `Composable` instances.

Operation	RE	GRE	WRE_w	SL
`concatenation`	$r_1 \cdot r_2$	$(p_1 \wedge p_2,\ r_1 \cdot r_2)$	`WSeq`	$P * Q$
`conjunction`	$r_1 \wedge r_2$	$(p_1 \wedge p_2,\ r_1 \wedge r_2)$	`WAnd`	$P \wedge Q$
`empty`	ε	$(\mathbf{true}, \varepsilon)$	`WEps sone`	**emp**
`universe`	$\neg\varnothing$	$(\mathbf{true}, \neg\varnothing)$	`wTop`	$\top$
`subtraction`	Brzozowski quotient	quotient (RE) + $\wedge$ (PPred)	`wSubtraction`	$P \mathrel{-\!\!*} Q$

The bind formula is unchanged: $\mathit{pre}(e \gg\!= f) = \mathit{pre}(e) \wedge (\mathit{post}(e) \mathrel{-\!\!*} \mathit{pre}(e_2))$ and $\mathit{future}(e \gg\!= f) = (\mathit{post}(e_2) \mathrel{-\!\!*} \mathit{future}(e)) \wedge \mathit{future}(e_2)$. Here $\mathrel{-\!\!*}$ is the magic wand (separating implication): $P \mathrel{-\!\!*} Q$ holds of heap h iff for every heap h' disjoint from h satisfying P, the combined heap $h \cup h'$ satisfies Q.

7.2 The SL Predicate Type

```
data SL
    = Emp              -- empty heap         (identity for SepStar)
    | Top              -- any heap           (identity for Conj)
    | Pure PPred       -- pure Presburger constraint (heap-independent)
    | Cell Addr Val    -- singleton: address a holds value v
    | SepStar SL SL    -- P * Q  separating conjunction
    | Conj SL SL       -- P ∧ Q ordinary conjunction
    | Wand SL SL       -- P -* Q magic wand (residual)
    deriving (Eq, Show)
```

7.3 Separating vs. Ordinary Conjunction

`SepStar` $P\ Q$ holds of heap h iff h splits into disjoint parts $h_1 \uplus h_2$ with $P(h_1)$ and $Q(h_2)$; it is the standard separating conjunction of ownership. `Conj` $P\ Q$ holds of h iff the *same* heap satisfies both P and Q. Without pure constraints, `Conj` on purely spatial predicates is either trivial ($\top \wedge P = P$) or unsatisfiable (`Cell` $l_1\ v_1 \wedge$ `Cell` $l_2\ v_2 =$ **false** for $l_1 \neq l_2$). The constructor becomes useful when one side is a pure constraint $\lceil \phi \rceil \wedge (\ell \mapsto v)$, combining a heap-independent condition with spatial ownership.

7.4 Presburger Arithmetic for Pure Constraints

Pure constraints reuse the same `PPred` language introduced in Section 5 (the `PExpr`/`PPred` types and `normalizePPred`). The predicate `Pure p` holds of any heap for which `evalPPred h p` is true; it enforces no ownership of cells. The smart constructor `pureSL` realises the standard SL box notation: `pureSL PTrue = Emp` and `pureSL p = Conj (Pure p) Emp` for non-trivial p.

7.5 The `Composable SL` Instance

```
instance Composable SL where
    concatenation = SepStar
    conjunction   = Conj
    empty         = Emp
    universe      = Top

    subtraction Emp q = q           -- emp -* Q = Q
    subtraction Top _ = Emp         -- Top trivially covers any pre
    subtraction p   q = Wand p q    -- general: symbolic wand
```

The two base cases mirror those of the RE instance: $\varepsilon \setminus r = r$ becomes $\mathbf{emp} \mathrel{-\!\!*} Q = Q$ (providing no heap leaves the obligation unchanged), and $\neg\varnothing \setminus r = \varepsilon$ becomes $\top \mathrel{-\!\!*} Q = \mathbf{emp}$ (a $\top$ postcondition covers any precondition, leaving no residual). The general case records the wand symbolically for later evaluation or normalisation.

7.6 Normalisation

The `normalizeSL` function applies the standard SL algebraic identities:

Law	Constructor
$\mathbf{emp} * Q = Q$, $P * \mathbf{emp} = P$	`SepStar`
$\top \wedge Q = Q$, $P \wedge \top = P$	`Conj`
$P \wedge P = P$ (idempotent)	`Conj`
$\mathbf{emp} \mathrel{-\!\!*} Q = Q$	`Wand`
$P \mathrel{-\!\!*} \top = \top$	`Wand`

All rules apply recursively bottom-up.

8 Examples

We demonstrate `Pledge` across ten use cases spanning all four instances. Each example defines primitive operations annotated with `pre`, `post`, and `future`, composes them in the `Pledge` monad, and inspects the normalised annotations of the result. A `future` of $\neg\varnothing$ (or `wNullable = sone` for WRE) and a `pre` of $\neg\varnothing$ together certify temporal correctness.

8.1 TLS Handshake Lifecycle

The `tls` library obligation is representative of a broad class of Haskell protocol obligations: a successful `handshake` must always be paired with a subsequent `bye`. We model the `Context` as an integer identifier.

```
type Context = Int

tlsHandshake :: Context → Pledge RE Context
tlsHandshake ctx = Pledge
```

```
    { ret    = ctx
    , pre    = universe
    , post   = Single (Atom "handshake" [Num ctx])
    , future = \c → finally (Atom "bye" [Num c]) }

tlsBye :: Context → Pledge RE ()
tlsBye ctx = Pledge
  { ret    = ()
  , pre    = Single (Atom "handshake" [Num ctx])
  , post   = Single (Atom "bye" [Num ctx])
  , future = \_ → universe }

tlsSend :: Context → Pledge RE ()
tlsSend ctx = Pledge
  { ret    = ()
  , pre    = Single (Atom "handshake" [Num ctx])
  , post   = Single (Atom "send" [Num ctx])
  , future = \_ → universe }

-- Good: handshake followed by data exchange then bye
goodSession = do
  ctx <- tlsHandshake 1
  tlsSend ctx
  tlsBye ctx

-- Bad future: bye omitted; residual = finally(bye(1))
missingBye = do
  ctx <- tlsHandshake 1
  tlsSend ctx

-- Bad future: two contexts opened, only one closed
partialClose = do
  ctx1 <- tlsHandshake 1
  ctx2 <- tlsHandshake 2
  tlsBye ctx1
```

The program `missingBye` retains `finally(bye(1))` as its residual future, naming the unclosed context precisely. Meanwhile, `partialClose` retains `finally(bye(2))`: the future condition is data-dependent, so the residual identifies context 2 specifically, not a generic "some context was not closed."

8.2 MVar Lifecycle

The `takeMVar` operation returns the taken handle; its data-dependent future condition uses the lambda `r` so the obligation tracks the *returned* handle. The `putMVar` operation carries as its future obligation `noUntil(putMVar(`v`), takeMVar(`v`))`, forbidding a second `putMVar(v)` until `takeMVar(v)` resets the guard. (The full definitions of `takeMVar` and `putMVar` appear in Section 3.2.)

```
-- Good: data-dependent  -  obligation discharged on the returned handle
takeAndPut n = do { v <- takeMVar n; putMVar v }
```

```
-- Good: re-acquire same MVar after releasing  -  noUntil is reset by takeMVar
takePutTakePut = do { v <- takeMVar 1; putMVar v; v <- takeMVar 1; putMVar v }

-- Bad future: putMVar(2) obligation remains (MVar 2 never returned)
missingPut = do { v1 <- takeMVar 1; putMVar v1; _ <- takeMVar 2; return () }

-- Bad pre: takeMVar(1) precondition not met
putWithoutTake = putMVar 1

-- Bad future: double-put immediately collapses future to Bot
doublePutImmediate = do { v <- takeMVar 1; putMVar v; putMVar v }

-- Bad future: double-put with intervening work, still collapses to Bot
doublePutWithWork = do
  { v1 <- takeMVar 1; v2 <- takeMVar 2; putMVar v1; putMVar v2; putMVar v1 }
```

The three examples below return non-unit values, showing that future conditions are tracked regardless of the return type.

```
-- Good: ret = MVar; future = finally(putMVar(1))  -  obligation visible in residual
takeReturnHandle :: Pledge RE MVar
takeReturnHandle = takeMVar 1

-- Bad future: ret = (MVar,MVar); future = finally(putMVar(1)) ∧ finally(putMVar(2))
takeTwoReturnPair :: Pledge RE (MVar, MVar)
takeTwoReturnPair = do { v1 <- takeMVar 1; v2 <- takeMVar 2; return (v1, v2) }

-- Good: ret = MVar; future = noUntil(putMVar(1),takeMVar(1))  -  no pending finally
takePutReturnHandle :: Pledge RE MVar
takePutReturnHandle = do { v <- takeMVar 1; putMVar v; return v }
```

8.3 Mutex / Lock Lifecycle

```
release :: Int → Pledge RE ()
release m = Pledge
  { ret = ()
  , pre    = Single (Atom "acquire" [Num m])
  , post   = Single (Atom "release" [Num m])
  , future = universe }

-- Bad future: lock 1 not released
lockLeak = do { acquire 1; acquire 2; release 2 }

-- Bad pre: release without acquire
releaseWithoutAcquire = release 1
```

8.4 Database Transactions

The `beginTx` operation carries a *disjunctive* future condition requiring that the transaction is eventually either committed or rolled back; `commit` and `rollback` each discharge this obligation:

```
beginTx :: Pledge RE ()
beginTx = Pledge
  { ret = (), pre = universe
  , post   = Single (Atom "beginTx" [])
  , future = Or (finally (Atom "commit" []))
                (finally (Atom "rollback" [])) }

commit :: Pledge RE ()
commit = Pledge
  { ret = (), pre = Single (Atom "beginTx" [])
  , post   = Single (Atom "commit" [])
  , future = universe }

rollback :: Pledge RE ()
rollback = Pledge
  { ret = (), pre = Single (Atom "beginTx" [])
  , post   = Single (Atom "rollback" [])
  , future = universe }

-- Bad future: transaction never closed
openTx = do { beginTx }

-- Good: transaction properly committed
committedTx = do { beginTx; commit }

-- Good: transaction properly rolled back
abortedTx = do { beginTx; rollback }
```

`openTx` retains the full disjunction `finally(commit)` ∨ `finally(rollback)` as its residual future. `committedTx` and `abortedTx` fully discharge the obligation, yielding ¬∅.

RE summary. Table 5 summarises the three RE domains.

Table 5. RE instance: bad programs and residual annotations.

Domain	Bad program	Residual `future`	Residual `pre`
Memory	`missingFree`	`finally(free(2))`	¬∅
Memory	`freeWithoutMalloc`	¬∅	∅
Memory	`doubleFreeImmediate`	∅	¬∅
Memory	`leakAndDoubleFree`	∅	¬∅
Memory	`allocReturnAddr`	`finally(free(1))`	¬∅
Memory	`allocTwoReturnPair`	`finally(free(1))` ∧ `finally(free(2))`	¬∅
Mutex	`acquire 1` leaked	`finally(release(1))`	¬∅
Mutex	release without acquire	¬∅	∅
Transaction	no commit/rollback	`finally(commit)` ∨ `finally(rollback)`	¬∅

8.5 Bounded Counter (GRE Instance)

The bounded counter (§5) demonstrates that both the Presburger and RE components are necessary. The RE alone would accept overflow sequences; the Presburger predicate alone would accept arbitrarily ordered events.

```
-- Good: init, two increments, one decrement, snapshot
normalUse :: Pledge GuardedRE ()
normalUse = do
  initCounter 0; increment 0 10; increment 0 10
  decrement 0;   snapshot 0

-- Bad: 11 increments against max 10
overflow :: Pledge GuardedRE ()
overflow = do
  initCounter 0
  sequence_ (replicate 11 (increment 0 10))
  snapshot 0
```

For `overflow`, the combined `pre` after composing all eleven increments normalises to $[\mathbf{false}] \wedge \neg\varnothing$: the Presburger side collapses to `PFalse` while the RE side remains the universal language, pinpointing the arithmetic violation.

GRE *summary.*

Bad program	Residual `pre` (GRE)	Cause
`overflow`	$[\mathbf{false}]\ \neg\varnothing$	$h[0] < 10$ violated
`underflow`	$[\mathbf{false}]\ \neg\varnothing$	$h[0] > 0$ violated
`noInit`	$[\mathbf{true}]\ \varnothing$	init event missing

8.6 Task Scheduling (WRE Instance)

The task-scheduling example shows that the tropical semiring naturally accumulates minimum costs. Each `submitJob` operation carries both a submission cost and a future condition requiring that every submitted job eventually completes.

```
-- Good: submit two jobs, complete both
goodSchedule :: Pledge (WRE Tropical) ()
goodSchedule = do
  submitJob 1 2.0; submitJob 2 3.0
  completeJob 1;   completeJob 2

-- Bad: job 2 never completed
missingComplete :: Pledge (WRE Tropical) ()
missingComplete = do
  submitJob 1 2.0; submitJob 2 3.0
  completeJob 1
```

For `goodSchedule`, `wNullable (evalFuture ...)` returns `Tropical 0`, indicating all obligations are discharged at zero residual cost. For `missingComplete`, the residual future still contains `wFinally (Tropical 0) (complete 2)`, and the overall weight is $+\infty$, flagging job 2 as unfinished.

The probability-semiring memory example follows analogously: a call `malloc a p` with $p = 0.9$ leaves a residual weight of 0.9 if `free a` is never called, dropping to 0 when it is.

8.7 Heap Memory (SL Instance)

The heap-memory example mirrors the RE memory example but uses heap ownership rather than event traces.

```
alloc :: Addr → Val → Pledge SL ()
alloc a v = Pledge
  { ret = (), pre = Top, post = Cell a v, future = Top }

free :: Addr → Val → Pledge SL ()
free a v = Pledge
  { ret = (), pre = Cell a v, post = Emp, future = Top }

writeCell :: Addr → Val → Val → Pledge SL ()
writeCell a old new = Pledge
  { ret = (), pre = Cell a old, post = Cell a new, future = Top }
```

Good: `alloc 0 1` ≫ `alloc 1 2`. The combined `post` is `SepStar (Cell 0 1) (Cell 1 2)`: two disjoint ownerships composed by separating conjunction, correctly reflecting that the two cells reside at distinct addresses.
Bad: leak (`alloc 0 1` ≫ `alloc 1 2` ≫ `free 0 1`). After freeing only address 0, the `post` retains `Cell 1 2`, naming the unreleased ownership.
Bad: read without ownership (`readCell 0 42`). The `pre` of the bare read is `Cell 0 42`, which is never discharged, flagging the missing ownership.

8.8 Bank Account with Presburger Guards (SL Instance)

The bank account example demonstrates `Pure` Presburger constraints inside `Conj`.

```
withdraw :: Addr → Val → Val → Pledge SL ()
withdraw a old amount = Pledge
  { ret = ()
  , pre  = Conj (Pure (PGe (ValAt a) (Lit amount))) (Cell a old)
  , post = Cell a (old - amount)
  , future = Top }

closeAccount :: Addr → Val → Pledge SL ()
closeAccount a bal = Pledge
  { ret = ()
  , pre  = Conj (Pure (PEq (ValAt a) (Lit 0))) (Cell a bal)
  , post = Emp, future = Top }
```

The precondition of `withdraw` is a `Conj` of a pure Presburger guard ($h[a] \geq$ `amount`) and a spatial ownership claim (`Cell` *a* `old`). Without the `Pure` constructor, `Conj` of two purely spatial predicates would be either trivial or unsatisfiable (see §7); the combination of `Pure` and `Conj` is what makes value-dependent preconditions expressible.
Bad: overdraft. After `deposit 0 0 100`, the account holds 100. The call `withdraw 0 100 150` carries *pre* = `Conj (Pure` ($h[0] \geq 150$`)) (Cell 0 100)`. Since $100 < 150$, the pure guard is not satisfiable, and the residual `pre` of the composed program retains this unsatisfiable constraint.

8.9 Linked List (SL Instance)

A singly-linked-list node occupies two consecutive cells: address a holds the payload value and address a+1 holds the next pointer.

```
allocNode :: Addr → Val → Val → Pledge SL ()
allocNode a val next = Pledge
  { ret = (), pre = Top
  , post = SepStar (Cell a val) (Cell (a+1) next)
  , future = Top }
```

Two-node list (`allocNode 0 10 2 ≫ allocNode 2 20 (-1)`). The combined `post` is a four-cell `SepStar`:

$$(\texttt{Cell}\ 0\ 10 * \texttt{Cell}\ 1\ 2) * (\texttt{Cell}\ 2\ 20 * \texttt{Cell}\ 3\ {-1})$$

This is the canonical representation of a two-node list in separation logic: all four cells are disjointly owned, and the structure of `SepStar` mirrors the structure of the list.

SL summary. Table 6 summarises the three SL domains. The key difference from the RE instance is that residual annotations are heap predicates rather than traces: a leaked cell appears in `post`, an unsatisfiable ownership constraint in `pre`.

Table 6. SL instance: bad programs and residual annotations.

Domain	Bad program	Residual `post`	Residual `pre`
Heap memory	`Cell 1 2` not freed	`Cell 1 2` (leaked)	$\top$
Heap memory	read without ownership	**emp**	`Cell 0 42`
Bank account	insufficient balance	`Cell 0` -50	unsatisfiable guard
Bank account	close with non-zero balance	**emp**	unsatisfied $h[0] = 0$
Linked list	read without ownership	**emp**	`Cell 0 99`

9 Monad Laws

Every monad must satisfy three equational laws that together guarantee sequential composition behaves predictably:

- **Left identity**: `pure x ≫= f = f x`. Wrapping a value in `pure` and immediately binding it is the same as applying `f` directly; `pure` introduces no effect of its own.
- **Right identity**: `e ≫= pure = e`. Binding a computation to `pure` is the same as the computation itself; `pure` consumes no observable state.
- **Associativity**: `(e ≫= f) ≫= g = e ≫= (\x → f x ≫= g)`. Re-parenthesising a chain of binds does not change its meaning; sequential composition is order-preserving but grouping-independent.

When these laws hold, programmers can freely refactor monadic chains without changing the observable semantics: extracting sub-expressions, fusing steps, or reordering `do`-blocks are all valid.

What the laws check. Because `future` has type `a → eff`, the laws are stated in terms of `evalFuture e = future e (ret e)`, written $\widehat{future}(e)$ for brevity. The laws hold for the triple $(ret, post, \widehat{future})$ assuming `Composable` forms a monoid under $\diamond$ with unit `empty`, and `universe` is the unit for `(∧\)`. The `pre` field satisfies the Hoare-rule property (Equation 4) rather than strict monad-law equality, because it accumulates residual precondition obligations; we return to this at the end of the section.

Left identity. `pure x >>= f = f x`.
ret: $ret(f\,x)$. ✓ *post*: $\varepsilon <> post(f\,x) = post(f\,x)$. ✓
For the future, let $fe = f(ret(\texttt{pure}\ x)) = f\,x$. Since $\widehat{future}(\texttt{pure}\ x) = \neg\varnothing$:

$$\widehat{future}(\texttt{pure}\ x \gg\!= f) = \big(post(fe) \setminus \widehat{future}(\texttt{pure}\ x)\big) \wedge \widehat{future}(fe)$$
$$= \big(post(f\,x) \setminus \neg\varnothing\big) \wedge \widehat{future}(f\,x) \;=\; \varepsilon \wedge \widehat{future}(f\,x) \;=\; \widehat{future}(f\,x).\ \checkmark$$

Right identity. `e >>= pure = e`.
ret: $ret(e)$. ✓ *post*: $post(e) <> \varepsilon = post(e)$. ✓
Let $fe = \texttt{pure}(ret(e))$, so $post(fe) = \varepsilon$ and $\widehat{future}(fe) = \neg\varnothing$. Then:

$$\widehat{future}(e \gg\!= \texttt{pure}) = \big(\varepsilon \setminus \widehat{future}(e)\big) \wedge \neg\varnothing \;=\; \widehat{future}(e) \wedge \neg\varnothing \;=\; \widehat{future}(e).\ \checkmark$$

Associativity. Re-parenthesising `(e >>= f) >>= g` as `e >>= (\x → f x >>= g)` must not change the specification.

ret and *post*: both sides yield $ret(g\,r_2)$ and $post(e) <> post(f\,r_1) <> post(g\,r_2)$, following from associativity of $\diamond$. ✓

For the future, let $r_1 = ret(e)$, $r_2 = ret(f\,r_1)$. Unfolding the bind formula (Equation (3)) on both sides and applying associativity of $\wedge$, both reduce to the same expression:

$$\big(post(g\,r_2) \setminus \big(post(f\,r_1) \setminus \widehat{future}(e)\big)\big) \;\wedge\; \widehat{future}(f\,r_1) \;\wedge\; \widehat{future}(g\,r_2).\ \checkmark$$

Precondition. The `pre` field satisfies the Hoare-rule property (Equation (4)) rather than strict monad-law equality. The residual $post(e) \setminus pre(e_2)$ captures which part of e_2's precondition e's postcondition has not discharged: full coverage gives the standard Hoare sequencing rule; absent coverage collapses to $\varnothing$, flagging the violation immediately.

10 Related Work

Pre/post-condition contracts in Haskell. Liquid Haskell [Vazou et al. 2014] expresses pre/post-conditions as refinement types verified by an SMT solver. Findler and Felleisen [Findler and Felleisen 2002] establish a runtime contract discipline for higher-order functions. QuickCheck [Claessen and Hughes 2000] supports conditional properties via the implication combinator `==>`, which restricts a property to inputs satisfying a given precondition. Swierstra and Baanen [Swierstra and Baanen 2019] embed Hoare triples as an indexed monad, propagating pre/post annotations through monadic bind analogously to `Pledge`. Pledge complements these by adding a *temporal* dimension through the future condition, without requiring type-system extensions.

Future conditions and temporal verification. ProveNFix [Song et al. 2024] encodes temporal properties as regular expressions to guide automated program repair. Pledge is complementary in orientation: rather than repairing programs after the fact, it carries temporal obligations as first-class annotations so that violations are identified through annotation inspection, prior to any effectful execution. Song et al. [2025] propose future conditions as a first-class extension to pre/post-condition contracts, formalising the notion that an operation may discharge temporal obligations onto its continuation. Pledge realises this idea as a Haskell library, contributing a monadic interface, derivative-based propagation, and four concrete `Composable` instances that the formal system leaves unaddressed.

Parameterised and graded monads. Atkey [2009] introduces parameterised monads, written $T(S_1, S_2, A)$, for Hoare-logic-style pre/post state tracking. Katsumata [2014] generalises this to

graded monads over a monoid; Orchard and Yoshida [2016] show effect systems and session types are instances. Pledge is related but orthogonal: rather than indexing by effect *type* at the type level, we carry an explicit obligation as `future :: a → eff` at the value level, preserving compatibility with the standard `Monad` class, **do**-notation, and the `mtl` stack.

Resource management in Haskell. The `bracket` combinator lexically scopes cleanup but cannot propagate obligations beyond the lexical scope. The `ResourceT` [Snoyman 2011] library threads a finaliser queue through every bind, discharging cleanup when the block exits. Both `Acquire` and `Managed` generalise this to an applicative/monadic interface. All three handle the *cleanup* half of a resource obligation but cannot express *protocol* or *quantitative* obligations, which require explicit future-condition propagation across bind steps.

Linear types. Linear Haskell [Bernardy et al. 2018] introduces multiplicity annotations enforced by GHC since version 9.0; `linear-base` [Tweag and contributors 2021] provides a `RIO` monad with linear file handles, statically guaranteeing every opened handle is consumed exactly once. Linear types enforce *structural* obligations (use exactly once), whereas future conditions enforce *temporal* obligations (use in a specified order, or with a specified probability). A hybrid combining linear types for uniqueness with Pledge for temporal ordering could achieve stronger guarantees than either alone.

Effect systems and algebraic effect handlers. Algebraic effect handlers [Pretnar 2015] decompose effectful computations into an operation set and a separate handler; resource lifecycle obligations are typically encoded in the handler's state machine. Polysemy's `Scoped` effect attaches resource creation and cleanup to a lexical scope threaded through the effect row. Pledge takes the complementary route: obligations are values in the annotation fields, composable across any handler.

Session types. Session types [Gay and Hole 2005; Honda 1993] enforce communication protocols at the type level. The `sessiontypes` library [van Walree 2017] realises this via an indexed monad `STTerm m s s' a`; the type checker rejects programs that violate the protocol order. Pledge is more lightweight: it operates within the standard `Monad` class, admits quantitative and heap-ownership obligations that session types do not, and is applicable beyond message-passing.

Typestate. Typestate systems [Bierhoff and Aldrich 2007; Strom and Yemini 1986] encode resource lifecycle protocols in the type system. Brady's `ST` monad encoding in Idris [Brady 2013] demonstrates full compile-time enforcement in a dependently typed language. Pledge achieves comparable expressiveness at the library level using standard Haskell type classes, at the cost of moving checking to annotation-inspection time rather than type-inference time: violations surface as non-trivial residual annotations in `pre` or `future`, which the programmer can evaluate before executing effects rather than relying on the type checker to reject programs.

Runtime monitoring. Runtime verification [Leucker and Schallhart 2009] checks temporal properties by instrumenting a concrete execution. Pledge propagates obligations in the specification layer: residuals surface as soon as annotations are composed, prior to any effectful run, without requiring a concrete execution.

Brzozowski derivatives. Derivatives of regular expressions [Brzozowski 1964] have found application in parsing [Might et al. 2011; Owens et al. 2009] and model checking. Equation (1) extends this to complement via $\partial_a(\neg r) = \neg(\partial_a r)$; our subtraction operator is a novel application to future-condition propagation in a monadic setting.

11 Discussion and Future Work

Decidability. Equality of normalised regular expressions is decidable, so the fixed-point computation in `subtraction` terminates whenever the derivative sequence is finite, which holds for the regular languages used in our examples. Extending to context-free or ω-regular specifications remains future work.

Precondition monad laws. The `pre` field satisfies the Hoare-rule property rather than strict monad-law equality (Section 9). A natural refinement is to make `pre` a separate, non-monadic annotation computed by a static analysis pass over the `Pledge` term, separating the behavioural monad from the precondition contract checker.

Integration with effect handlers. Lifting Pledge into an effect-handler framework (`effectful` or `polysemy`) would allow temporal specifications to compose alongside algebraic effect rows. A lightweight *shadow* approach requires no deep integration: keep the `Pledge RE` computation as a pure specification evaluated as obligation values before the run, while a separate `Eff es` computation carries out the real effects, with assertions linking the two at the top level:

```
main = do
  assert (normalize (pre        spec) == top) "precondition␣violated"
  assert (normalize (evalFuture spec) == top) "future␣obligation␣unmet"
  runEff impl
```

Annotation checking is entirely pure; no changes to the effect stack are required.

Closed-world alphabet assumption. The `firstWith` function uses $\Sigma_0 = atoms(r_1) \cup atoms(r_2)$ as its working alphabet. This is sound under the *closed-world assumption* that every relevant event is mentioned in the two operands. A fully open-world implementation would accept an explicit Σ declaration, enabling complement over an unrestricted domain.

Connection to indexed and graded monads. Because `future` :: $a \to eff$ indexes the obligation by the return value, Pledge echoes the indexed/graded-monad literature where types are indexed by computational effects; here the index is a return value instead of an effect row.

Specification versus runtime enforcement. Pledge is a *specification* tool: the `pre`, `post`, and `future` fields are pure Haskell values that describe what a computation *should* do, not runtime guards that intercept execution. In particular, Pledge does not integrate with GHC's asynchronous-exception machinery. If an async exception fires mid-computation and the continuation never calls `putMVar`, Pledge's `future` condition correctly flags the *specification* as carrying an unmet obligation, but no runtime mechanism prevents the violation from occurring. Pairing Pledge specifications with runtime enforcement via `bracket` or `mask` (so that obligations are both expressed and enforced) remains future work.

Solver-backed discharge of residual conditions. Residual annotations are currently surfaced as symbolic expressions. A natural extension would connect them to an automatic solver: RE residuals checked by NFA/DFA construction or a BDD library; SL predicates with Presburger guards discharged by Z3 or CVC5, reporting concrete counterexample traces or unsatisfiable heaps.

Parallel composition. The `Pledge` monad is inherently sequential; concurrent programs require a parallel combinator $e_1 \parallel e_2$. In the RE instance this corresponds to the shuffle product $r_1 \sqcup\!\sqcup r_2$; the SL instance admits a parallel connective via the concurrent separation logic frame rule [O'Hearn 2007]. Extending `Composable` with a `parallel` operation is a natural next step.

Quantitative future conditions. A natural further direction is to connect WRE to probabilistic temporal logics (PCTL) or quantitative Kleene algebras; in the SL instance, a quantitative analogue could express amortised resource bounds or expected heap usage.

12 Conclusion

We have presented *Pledge*, a Haskell library that augments effectful computations with *future conditions*: annotations that constrain what the remainder of a program must accomplish.

The central insight is that trace-based, arithmetic, quantitative, and heap-ownership obligations are structurally identical at the level of a five-operation `Composable` algebra. All four instances (RE, GRE, WRE, SL) share a single monadic bind formula, and the diagnostic signal is uniform: a non-trivial residual in `pre` or `future` identifies exactly which obligation remains unmet and why.

Pledge requires no language extensions beyond standard type classes, composes via a standard `Monad` instance, and integrates transparently with the `mtl` stack. Immediate directions for future work include parallel composition (shuffle product in the RE instance; frame rule in the SL instance), solver-backed residual discharge, and integration with algebraic effect handlers.

References

Robert Atkey. 2009. Parameterised Notions of Computation. *Journal of Functional Programming* 19, 3–4 (2009), 335–376. doi:10.1017/S095679680900728X

Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. Linear Haskell: Practical Linearity in a Higher-Order Polymorphic Language. *Proceedings of the ACM on Programming Languages* 2, POPL, Article 5 (2018), 29 pages. doi:10.1145/3158093

Kevin Bierhoff and Jonathan Aldrich. 2007. Modular Typestate Checking of Aliased Objects. In *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications (OOPSLA)*. 301–320. doi:10.1145/1297027.1297050

Edwin Brady. 2013. Idris, a General-Purpose Dependently Typed Programming Language: Design and Implementation. *Journal of Functional Programming* 23, 5 (2013), 552–593. doi:10.1017/S095679681300018X

Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (1964), 481–494. doi:10.1145/321239.321249

Koen Claessen and John Hughes. 2000. QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs. In *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 268–279. doi:10.1145/351240.351266

Robert Bruce Findler and Matthias Felleisen. 2002. Contracts for Higher-Order Functions. In *Proceedings of the 7th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 48–59. doi:10.1145/581478.581484

Simon J. Gay and Malcolm Hole. 2005. Subtyping for Session Types in the Pi Calculus. *Acta Informatica* 42, 2-3 (2005), 191–225. doi:10.1007/s00236-005-0177-z

Kohei Honda. 1993. Types for Dyadic Interaction. In *Proceedings of the 4th International Conference on Concurrency Theory (CONCUR) (Lecture Notes in Computer Science, Vol. 715)*. 509–523. doi:10.1007/3-540-57208-2_35

Shin-ya Katsumata. 2014. Parametric Effect Monads and Semantics of Effect Systems. *ACM SIGPLAN Notices* 49, 1 (2014), 633–646. doi:10.1145/2578855.2535846

Martin Leucker and Christian Schallhart. 2009. A Brief Account of Runtime Verification. *Journal of Logic and Algebraic Programming* 78, 5 (2009), 293–303. doi:10.1016/j.jlap.2008.08.004

Matthew Might, David Darais, and Daniel Spiewak. 2011. Parsing with Derivatives: A Functional Pearl. In *Proceedings of the 16th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 189–195. doi:10.1145/2034773.2034801

Peter W. O'Hearn. 2007. Resources, Concurrency, and Local Reasoning. *Theoretical Computer Science* 375, 1–3 (2007), 271–307. doi:10.1016/j.tcs.2006.12.035

Dominic Orchard and Nobuko Yoshida. 2016. Effects as Sessions, Sessions as Effects. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*. 568–581. doi:10.1145/2837614.2837634

Scott Owens, John Reppy, and Aaron Turon. 2009. Regular-expression Derivatives Re-examined. *Journal of Functional Programming* 19, 2 (2009), 173–190. doi:10.1017/S0956796808007090

Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. Invited tutorial paper. *Electronic Notes in Theoretical Computer Science* 319 (2015), 19–35. doi:10.1016/j.entcs.2015.12.003

Michael Snoyman. 2011. conduit: Streaming Data Processing. https://hackage.haskell.org/package/conduit. Haskell package on Hackage; introduces the `ResourceT` transformer.

Yahui Song, Darius Foo, and Wei-Ngan Chin. 2025. Specifying and Verifying Future Conditions. In *Static Analysis — 32nd International Symposium, SAS 2025 (Lecture Notes in Computer Science, Vol. 16100)*. Springer, 90–112. doi:10.1007/978-3-032-07106-4_5

Yahui Song, Xiang Gao, Wenhua Li, Wei-Ngan Chin, and Abhik Roychoudhury. 2024. ProveNFix: Temporal Property-Guided Program Repair. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 11 (2024), 23 pages. doi:10.1145/3643737

Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* 12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929

Wouter Swierstra and Tim Baanen. 2019. A Predicate Transformer Semantics for Effects (Functional Pearl). *Proceedings of the ACM on Programming Languages* 3, ICFP, Article 103 (2019), 26 pages. doi:10.1145/3341707

Tweag and contributors. 2021. linear-base: Standard Library for Linear Types in Haskell. https://github.com/tweag/linear-base.

Ferdinand van Walree. 2017. sessiontypes: Session Types for Haskell. https://hackage.haskell.org/package/sessiontypes.

Niki Vazou, Eric L. Seidel, Ranjit Jhala, Dimitrios Vytiniotis, and Simon Peyton Jones. 2014. Refinement Types for Haskell. In *Proceedings of the 19th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. 269–282. doi:10.1145/2628136.2628161